

U1T02: Document Intelligence Async Pipeline

Russel Emmanuel Ku Aguilar
Data Engineering
Universidad Politécnica de Yucatán
Km. 4.5. Carretera Mérida — Tetiz
Tablaje Catastral 4448. CP 97357
Ucú, Yucatán. México
Email: 2309131@upy.edu.mx

Dexter Gomez.
Universidad Politécnica de Yucatán
Km. 4.5. Carretera Mérida — Tetiz
Tablaje Catastral 4448. CP 97357
Ucú, Yucatán. México
Email: gomez.dexter@upy.edu.mx

Abstract

This paper presents an enterprise-grade asynchronous document intelligence pipeline designed to process document uploads reliably while avoiding the limitations of synchronous HTTP processing. The proposed architecture implements the Claim-Check Pattern by persisting uploaded binary files in shared storage and transmitting only lightweight job identifiers through a Redis message broker. FastAPI manages document ingestion and job tracking, while Celery workers perform asynchronous extraction using a format-aware strategy based on PyMuPDF, pypdf, Tesseract OCR, and python-docx. PostgreSQL 16 provides persistent job lifecycle management, metadata storage, extracted content, and error tracking, while Celery Flower enables real-time monitoring of asynchronous tasks. The system also incorporates fault-tolerance mechanisms, including late task acknowledgments, worker recovery, corrupted-file handling, and upload-size validation. An empirical benchmark was conducted using a two-page PDF document to compare PyMuPDF, pypdf, and Tesseract OCR in terms of extraction time and output. The resulting containerized architecture can be deployed through Docker Compose and provides a reproducible environment for asynchronous document processing, monitoring, and result retrieval.

Index Terms

asynchronous processing, document intelligence, Claim-Check Pattern, FastAPI, Celery, Redis, PostgreSQL, OCR, PyMuPDF, Docker Compose



CONTENTS

I	Introduction	2
II	System Architecture	2
II-A	Architectural Overview	2
II-B	Claim-Check Pattern	2
II-C	Ingestion and Processing Flow	2
III	Technology Stack	2
IV	System Interfaces	3
IV-A	REST API	3
IV-B	Web Dashboard	4
IV-C	Celery Flower Monitoring	4
V	Technical Design Decisions	4
V-A	Claim-Check Pattern versus Direct Queue Payload	4
V-B	PostgreSQL versus SQLite and NoSQL	4
V-C	Celery and Redis	4
V-D	Multi-Parser Document Intelligence Engine	4
VI	Empirical Parser Benchmark	4
VII	Resilience and Fault Tolerance	5
VIII	Database Design	5
VIII-A	Lifecycle States	5
IX	Deployment and Execution	5
X	Repository Structure	5
XI	Verification and Results	5
XII	Conclusion	6
	References	6

U1T02: Document Intelligence Async Pipeline

““latex

I. INTRODUCTION

Real-world document processing systems cannot reliably process file uploads synchronously. Documents arrive unpredictably, vary significantly in file size and format, and may require CPU-intensive parsing and Optical Character Recognition (OCR). Maintaining an HTTP connection during long-running document processing can lead to network timeouts, server socket exhaustion, and poor user experience.

This project implements an enterprise-grade *Document Intelligence Asynchronous Pipeline* designed to decouple document ingestion from document processing. The system accepts document uploads, persists raw files, creates asynchronous processing jobs, executes extraction tasks in background workers, and allows users to monitor job progress and retrieve extracted text and metadata.

The architecture follows the *Claim-Check Pattern*, in which the complete binary document is stored in persistent shared storage while only a lightweight job identifier is transmitted through the message broker. This approach prevents large binary payloads from occupying message-queue memory and allows the system to process multiple jobs asynchronously.

The complete solution is containerized using Docker Compose and consists of five coordinated services: a FastAPI ingestion service, a Celery worker, a Redis message broker, a PostgreSQL 16 database, and a Celery Flower monitoring interface. The complete stack can be started using a single Docker Compose command [1].

II. SYSTEM ARCHITECTURE

A. Architectural Overview

The proposed architecture separates document ingestion, persistent storage, asynchronous execution, database tracking, and monitoring into independent components. This separation prevents the API layer from remaining blocked while computationally expensive document extraction operations are executed.

The main technologies used by the system are FastAPI and Uvicorn for the REST API, Celery for asynchronous task execution, Redis 7.2 as the message broker and result backend, PostgreSQL 16 for job tracking and metadata, Tesseract OCR for scanned documents and images, and specialized parsers such as PyMuPDF, pypdf, and python-docx [1], [2].

B. Claim-Check Pattern

The central architectural decision is the implementation of the Claim-Check Pattern. Instead of placing complete binary files inside Redis messages, the API first stores the uploaded document in the shared storage volume. The resulting storage

TABLE I
TECHNOLOGY STACK

Component	Technology	Version	Purpose
API Framework	FastAPI / Uvicorn	0.110+	Asynchronous REST API
Task Queue	Celery	5.3+	Background task execution and retries
Message Broker	Redis	7.2-alpine	Message broker and result backend
Database	PostgreSQL	16-alpine	Job tracking and metadata
ORM	SQLAlchemy	2.0+	Database models and connection management
OCR Engine	Tesseract OCR	5.0+	Spanish and English OCR
PDF Parser	PyMuPDF / pypdf	1.23+	PDF text extraction
DOCX Parser	python-docx	1.1+	Word document extraction
Monitoring	Celery Flower	2.0+	Worker and queue monitoring
Containerization	Docker Compose	v2/v5	Service orchestration

location is associated with a unique `job_id`. Only this lightweight identifier is then submitted to Redis.

This approach prevents large files, such as 25 MB PDF documents, from being serialized and transmitted through the message broker. Consequently, Redis messages remain small while the binary document remains in persistent storage. According to the project architecture, the claim-check message can remain below approximately 1 KB, allowing the broker to handle a large number of queued jobs without unnecessarily consuming memory [2].

C. Ingestion and Processing Flow

The complete ingestion process consists of twelve logical operations. The client first submits a document through the upload endpoint. FastAPI writes the binary file to shared storage and creates a corresponding database record with a `PENDING` status. The API then places the job identifier into Redis and immediately returns an `HTTP 202 Accepted` response.

The Celery worker consumes the identifier, retrieves the binary document from shared storage, updates the database state to `PROCESSING`, executes the appropriate extraction strategy, stores the resulting text and metadata, and finally marks the job as `COMPLETED`. The client can independently query the job status using the corresponding job identifier.

III. TECHNOLOGY STACK

Table I summarizes the principal technologies used in the implementation.

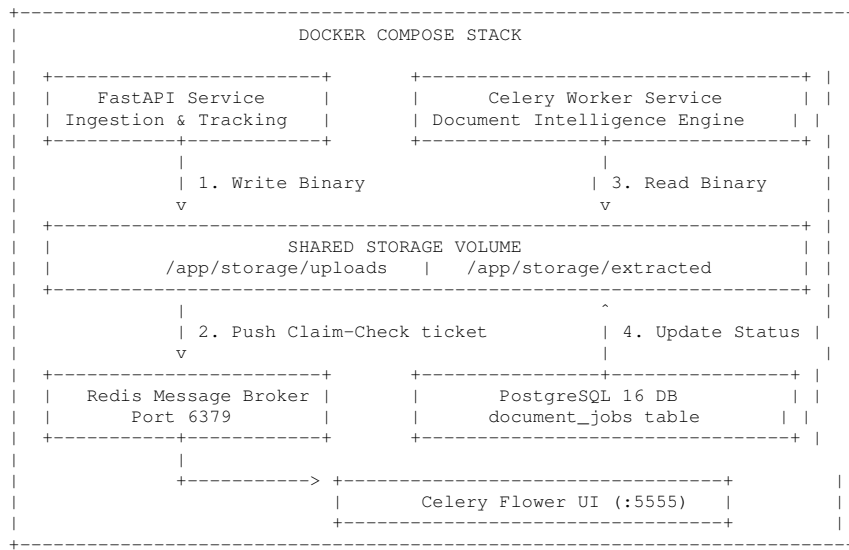


Fig. 1. High-level architecture of the asynchronous document intelligence pipeline.

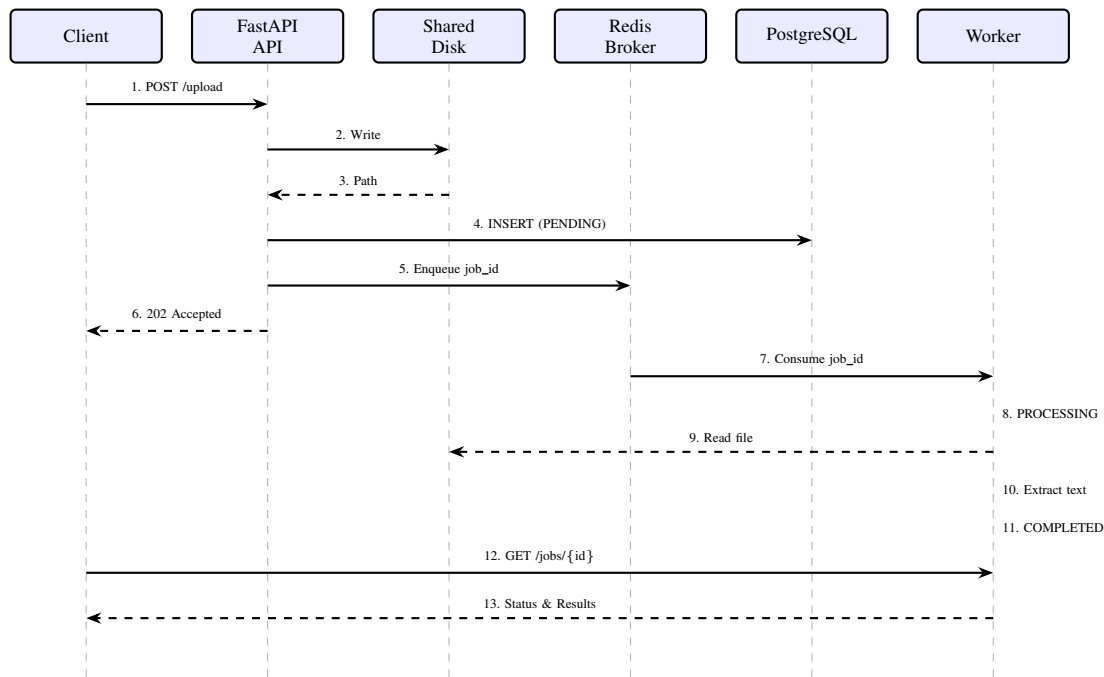


Fig. 2. Document ingestion and asynchronous processing sequence.

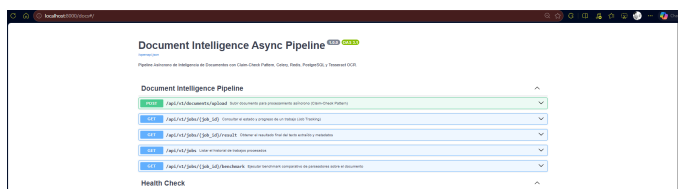
IV. SYSTEM INTERFACES

A. REST API

The REST API exposes endpoints for document ingestion, job tracking, result retrieval, benchmarking, and service health verification. Upload operations return immediately with an HTTP 202 Accepted response, allowing document extraction to continue asynchronously.

TABLE II
REST API ENDPOINTS

Method	Endpoint	Description
POST	/api/v1/documents/upload	Upload document for asynchronous processing
GET	/api/v1/jobs/{job_id}	Retrieve status, progress, and duration
GET	/api/v1/jobs/{job_id}/results	Retrieve extracted text and metadata
GET	/api/v1/jobs	List recent jobs with pagination
GET	/api/v1/jobs/{job_id}/benchmark	Execute parser benchmark
GET	/health	Verify service health



B. Web Dashboard

The web dashboard provides an interactive interface for uploading documents and monitoring asynchronous processing jobs. The interface communicates with the FastAPI service while the actual document-processing operations are performed independently by Celery workers.

The dashboard exposes the state of document jobs and allows users to retrieve processing results without maintaining a long-lived upload connection.

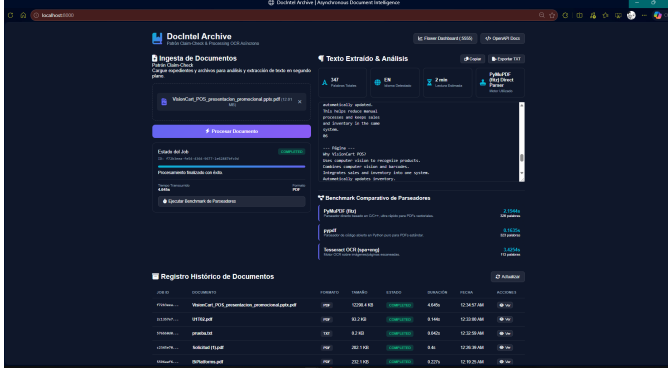


Fig. 4. Interactive web dashboard for document ingestion and job tracking.

C. Celery Flower Monitoring

Celery Flower provides a monitoring interface for the asynchronous processing layer. It exposes information about workers, tasks, queue activity, and task execution, allowing the asynchronous pipeline to be observed independently from the main web interface.

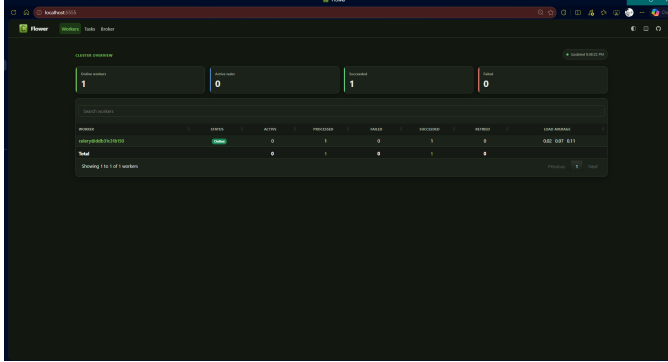


Fig. 5. Celery Flower interface for monitoring asynchronous workers and tasks.

V. TECHNICAL DESIGN DECISIONS

A. Claim-Check Pattern versus Direct Queue Payload

The system uses the Claim-Check Pattern rather than transmitting binary documents directly through Redis. This decision is motivated by the size and nature of document files. A direct queue payload would require serialization of complete binary files and would increase broker memory consumption and message-processing overhead.

With the selected architecture, Redis receives only the `job_id`, while the actual document remains in shared storage. This reduces message size and keeps the message broker focused on coordination rather than binary storage.

TABLE III
DOCUMENT EXTRACTION STRATEGY

Format	Primary Strategy	Fallback Strategy
Vectorial PDF	PyMuPDF (<i>fitz</i>)	pypdf
Scanned PDF	PyMuPDF rendering	Tesseract (<i>spa+eng</i>)
PNG/JPG	OpenCV/PIL preprocessing	Tesseract (<i>spa+eng</i>)
DOCX	python-docx	Text buffer
TXT/MD	Multi-encoding reader	UTF-8 fallback

B. PostgreSQL versus SQLite and NoSQL

PostgreSQL 16 was selected as the database layer and is accessed through SQLAlchemy 2.0. The database stores document-job state, metadata, processing timestamps, extracted text, errors, and retry information.

SQLite was not selected because the expected concurrent workload involves Celery workers modifying job records while API requests simultaneously poll job states. The project report identifies SQLite's file-level locking behavior as a potential source of database is locked errors under these conditions.

PostgreSQL provides transactional consistency, concurrent access, row-level locking, and native JSON support for extracted document metadata.

C. Celery and Redis

Celery was selected as the distributed task queue and Redis 7.2 as its broker and result backend. The implementation uses asynchronous execution, retries, rate-limiting capabilities, late acknowledgments, and Flower integration.

The configuration `task_acks_late = True` is particularly important for fault tolerance because the message is not acknowledged as completed until the worker finishes the task.

D. Multi-Parser Document Intelligence Engine

The document extraction engine uses a format-aware strategy instead of applying a single parser to every document.

For native PDFs, PyMuPDF is used to extract embedded vector text directly. Scanned documents are rendered into images and processed through Tesseract OCR. Image files use preprocessing followed by OCR. DOCX documents are parsed structurally to retrieve paragraphs and table contents, while TXT and Markdown files are decoded using multiple encoding strategies.

VI. EMPIRICAL PARSER BENCHMARK

The implementation includes a programmatic benchmark designed to compare extraction strategies using the assignment document `U1T02.pdf`, consisting of two pages and approximately 433 words.

The benchmark demonstrates that PyMuPDF provides fast extraction for native text PDFs while requiring substantially less processing time than OCR. The measured Tesseract execution time was approximately 1.747 seconds compared with 0.087 seconds for PyMuPDF on the evaluated document.

TABLE IV
PARSER BENCHMARK RESULTS

Parser	Time	Chars.	Words	Memory
PyMuPDF	0.087 s	2,929	430	~12 MB
pypdf	0.029 s	2,927	430	~8 MB
Tesseract OCR	1.747 s	2,900	430	~85 MB

TABLE V
EDGE CASES AND MITIGATION STRATEGIES

Edge Scenario	System Behavior
Unsupported format	Rejected at the API boundary with HTTP 400 before storage or queue dispatch.
Corrupted file	Worker catches the exception and updates the job to FAILED with an error message.
Worker failure	Late task acknowledgment allows Redis to redeliver the task.
Container crash	Service startup checks stale processing jobs and performs recovery handling.
Large file	Upload is streamed and rejected with HTTP 413 when exceeding the configured limit.

Tesseract remains necessary for scanned documents and images because these documents do not necessarily contain an embedded text stream. Therefore, the architecture does not treat the benchmark as a single-parser selection problem; instead, it uses different extraction mechanisms according to document characteristics.

The implementation dynamically evaluates extracted character density. If PyMuPDF extracts fewer than approximately 15 characters per page, the pipeline activates the Tesseract OCR fallback mechanism.

VII. RESILIENCE AND FAULT TOLERANCE

The pipeline incorporates several mechanisms for handling abnormal execution conditions.

The Celery configuration uses late task acknowledgment so that a task is not removed from the broker before successful completion. If a worker terminates during processing, the task can therefore be made available again.

The database also records timestamps such as `started_at`, allowing stale processing jobs to be identified after service restarts. Exceptions generated during document processing are persisted in the corresponding job record rather than leaving jobs indefinitely in a non-terminal state.

VIII. DATABASE DESIGN

The central database table is `document_jobs`. It stores document identity, storage information, lifecycle state, progress, extracted content, metadata, error information, retry count, and execution timestamps.

A. Lifecycle States

The document lifecycle begins when the upload operation creates a `PENDING` job. Once a Celery worker begins execution, the state changes to `PROCESSING`. During processing, the system reports progress associated with reading, extraction, and metadata operations.

The final state is either `COMPLETED` or `FAILED`.

TABLE VI
DOCUMENT JOB DATA MODEL

Field	Purpose
<code>id</code>	Unique job identifier
<code>filename</code>	Original document filename
<code>file_format</code>	Detected document format
<code>file_size_bytes</code>	Binary file size
<code>storage_path</code>	Shared-storage location
<code>status</code>	Current lifecycle state
<code>progress_percent</code>	Processing progress
<code>status_message</code>	Human-readable processing status
<code>extracted_text</code>	Extracted document content
<code>extracted_metadata</code>	Structured document metadata
<code>error_message</code>	Processing error information
<code>retry_count</code>	Number of processing retries
<code>created_at</code>	Job creation timestamp
<code>started_at</code>	Processing start timestamp
<code>completed_at</code>	Processing completion timestamp
<code>processing_duration_seconds</code>	Total processing duration

IX. DEPLOYMENT AND EXECUTION

The entire system is containerized and orchestrated using Docker Compose. The repository provides separate images for the API and Celery worker, while PostgreSQL and Redis provide the persistence and messaging layers.

The complete stack can be started using:

```
[language=bash,caption=Docker Compose startup command,label=lst:docker] docker compose up --build -d
```

The documented service interfaces are:

- Web Dashboard: `http://localhost:8000`
- OpenAPI / Swagger: `http://localhost:8000/docs`
- Celery Flower: `http://localhost:5555`
- PostgreSQL: `localhost:5432`

The repository also includes an automated end-to-end test suite, `test_all_formats.py`, for validating supported document formats.

X. REPOSITORY STRUCTURE

The project repository is organized into infrastructure, application, storage, and testing components: The `app` directory contains the FastAPI entry point, configuration, API routers, Claim-Check storage logic, SQLAlchemy models, request and response schemas, extraction services, Celery task definitions, and static web interface components.

XI. VERIFICATION AND RESULTS

The implementation was verified through the complete Docker Compose deployment and the automated document-format test suite. According to the project documentation, the system was successfully executed using the single-command Docker Compose orchestration and the end-to-end test suite reported a 100% pass rate for the supported formats [1], [2].

The resulting system provides an asynchronous processing architecture in which ingestion is separated from computationally intensive extraction. The API can therefore acknowledge uploads without waiting for OCR or document parsing to finish.

The architecture also provides monitoring through Celery Flower, persistent job tracking through PostgreSQL, and shared binary storage for the Claim-Check implementation.

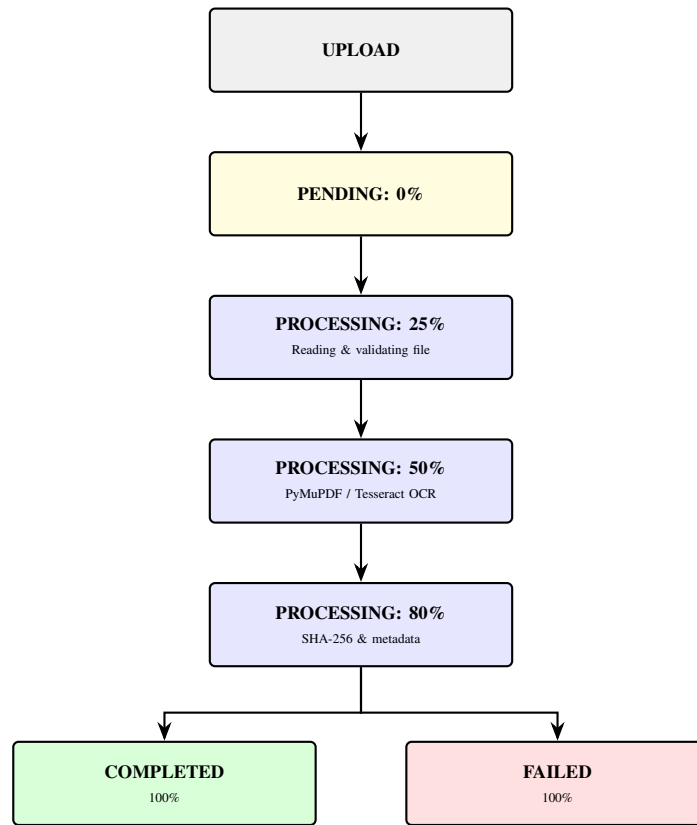


Fig. 6. Document job lifecycle and state transitions.

```

Document-Intelligence-Async-Pipeline/
|-- docker-compose.yml
|-- Dockerfile.api
|-- Dockerfile.worker
|-- requirements.txt
|-- README.md
|-- test_all_formats.py
|-- docs/
|   |-- REPORT.md
|   '-- REPORT.pdf
|-- storage/
|   |-- uploads/
|   '-- extracted/
'-- app/
    |-- main.py
    |-- config.py
    |-- api/v1/
    |-- core/
    |-- db/
    |-- schemas/
    |-- services/
    |-- tasks/
    '-- static/
  
```

Fig. 7. Repository structure.

XII. CONCLUSION

This project implements a fault-tolerant asynchronous document intelligence pipeline based on the Claim-Check Pattern. The architecture separates binary storage, message-based coordination, document processing, database state management, and operational monitoring into independent services.

The use of Redis and Celery allows document extraction to execute asynchronously, while PostgreSQL provides persistent lifecycle tracking. The format-aware parser strategy combines PyMuPDF, pypdf, Tesseract OCR, python-docx, and multi-encoding readers to support different document types.

The empirical benchmark provides quantitative evidence for selecting different extraction mechanisms according to document characteristics. PyMuPDF is used for native PDF extraction, while Tesseract provides the required OCR capability for scanned and image-based documents.

Finally, the Docker Compose deployment provides a reproducible execution environment in which the complete architecture can be started through a single command. The combination of asynchronous processing, persistent job state, fault recovery, and monitoring establishes the foundation for a scalable document-processing service.

REFERENCES

- [1] R. E. Ku Aguilar, "Document Intelligence Asynchronous Pipeline," GitHub repository, Sep. 2026. [Online]. Available: <https://github.com/RusselKu/Document-Intelligence-Async-Pipeline>
- [2] Technical Architecture Team, "Technical Architecture & System Report: Document Intelligence Asynchronous Pipeline (U1T02)," Sep. 2026.